

Task 3 - DevOps CRM Project Setup and Automation

1. Objective

The objective of this task was to clone and understand the DevOps CRM project, set up and run the application locally, and create a Python script to automate the local setup and startup process.

The automation was implemented using Python as required, without using a shell script (.sh). An additional HTTP health check was included to verify that the application was actually reachable after startup.

2. Project Repository

Repository:

<https://github.com/PearlThoughts-Intern-DevOps/devops-crm-project>

3. Project Exploration

The repository was cloned to the local system and the project structure was explored.

Important files and directories include:

- src/ - Main application source code.
- public/ - Public application resources.
- .github/ - GitHub-related configuration and workflows.
- package.json - Project information, dependencies, and available scripts.
- yarn.lock - Locked dependency versions.
- .nvmrc - Required Node.js version.
- SETUP.md - Local setup instructions.
- README.md - General project information.

4. Environment Setup

The project required Node.js, Yarn, Docker, and Python.

The project uses Node.js version 24.5.0 according to the .nvmrc file. Yarn was managed using Corepack. Docker Desktop was installed and configured with the WSL 2 backend.

The required tools and Docker environment were verified before running the project.

5. Installing Dependencies

Project dependencies were installed using:

```
yarn.cmd install
```

The installation completed successfully. Some peer dependency warnings were displayed, but they did not prevent the installation from completing.

6. Running the Application

The Twenty development environment was started using:

```
yarn.cmd twenty docker:start
```

The Twenty Docker image was pulled and the application was started in the Docker container.

The application became available at:

```
http://localhost:2020
```

The running container was verified using:

```
docker ps
```

The Twenty server status was also verified using:

```
yarn.cmd twenty docker:status
```

7. Python Automation

A Python script named `setup_and_start.py` was created to automate the local setup and startup process.

The script performs the following operations:

1. Checks required tools

The script verifies the availability of Node.js, Docker, and Yarn.

2. Checks Docker health

The script verifies that the Docker engine is running correctly before attempting to start the application.

3. Installs dependencies

The script runs `yarn.cmd install` to install the required project dependencies.

4. Starts the Twenty application

The script runs `yarn.cmd twenty docker:start` to start the Twenty server.

5. Verifies Twenty server status

The script runs `yarn.cmd twenty docker:status` to verify that the Twenty server is running and healthy.

6. Performs an HTTP health check

The script sends a request to `http://localhost:2020` and verifies that the application responds successfully.

8. HTTP Health Check

The HTTP health check was added as an extra validation step in the Python automation.

A running Docker container alone does not always guarantee that the application is fully accessible. Therefore, after starting the application, the script checks whether the application URL responds successfully.

During testing, the application returned HTTP 200.

This confirmed that the application was running and accessible through the local URL.

9. Automation Testing

The automation was tested in different scenarios.

Scenario 1: Application already running

When the application was already running, the Twenty CLI detected the existing server. No duplicate container was created, and the health checks completed successfully.

Scenario 2: Existing container stopped

The Twenty server was stopped using:

```
yarn.cmd twenty docker:stop
```

After stopping it, `docker ps` showed no running containers. The Python automation was then executed.

The script started the existing stopped container and completed the application startup process. The HTTP health check then returned HTTP 200.

The container ID remained the same, confirming that the existing container was restarted rather than a duplicate container being created.

10. Important Understanding: `docker ps` vs `docker ps -a`

The following difference was observed during testing:

`docker ps`

Shows only currently running containers.

`docker ps -a`

Shows both running and stopped containers.

After stopping the Twenty server, the container was no longer running but still existed. Therefore, when the automation was executed again, the startup command displayed that it was starting the existing container.

11. Issues Faced and Solutions

Issue 1: Development tools were not available in the MSYS2 UCRT64 environment.

Solution: PowerShell was used because Node.js, Python, Docker, and Git were available in the Windows environment.

Issue 2: PowerShell execution policy blocked direct npm/yarn PowerShell scripts.

Solution: The .cmd version of Yarn was used, for example: `yarn.cmd install`.

Issue 3: Docker was not initially available.

Solution: WSL 2 was installed and Docker Desktop was installed and configured successfully.

Issue 4: Confusion between a stopped container and a deleted container.

Solution: The difference between `docker ps` and `docker ps -a` was verified. Stopping the server stopped the container but did not remove it.

12. Result

The DevOps CRM project was successfully set up and run locally.

A Python automation script was successfully created to automate:

- Required tool verification
- Docker health verification
- Dependency installation
- Application startup
- Twenty server status verification
- HTTP application health verification

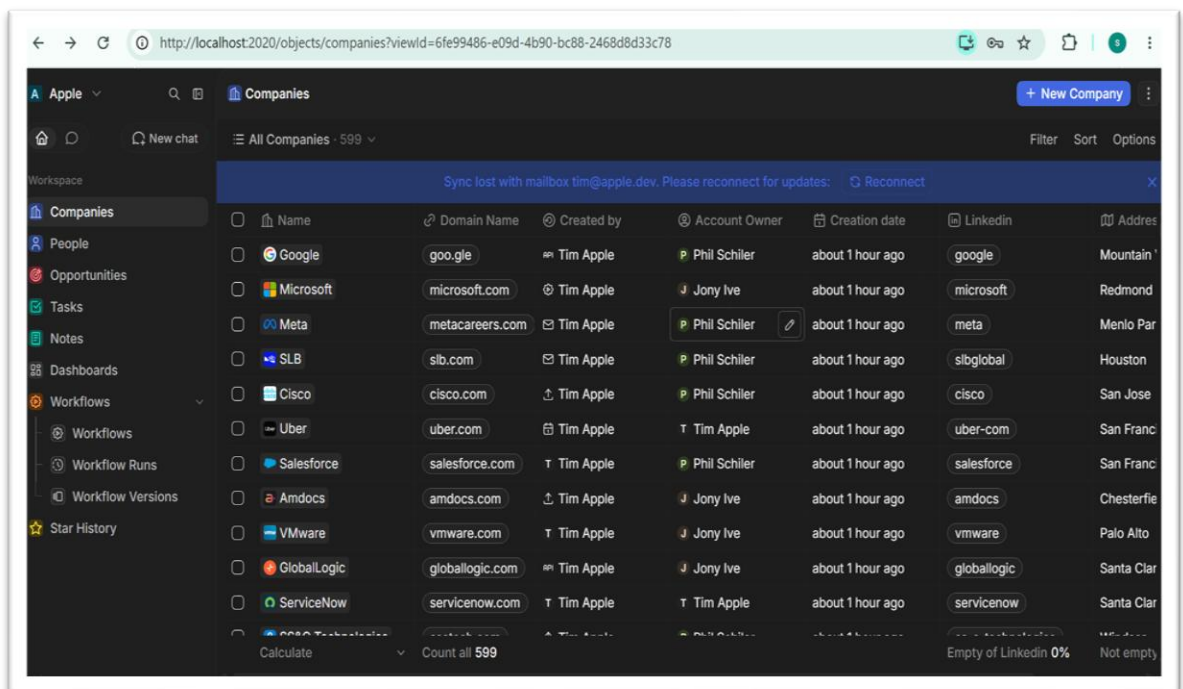
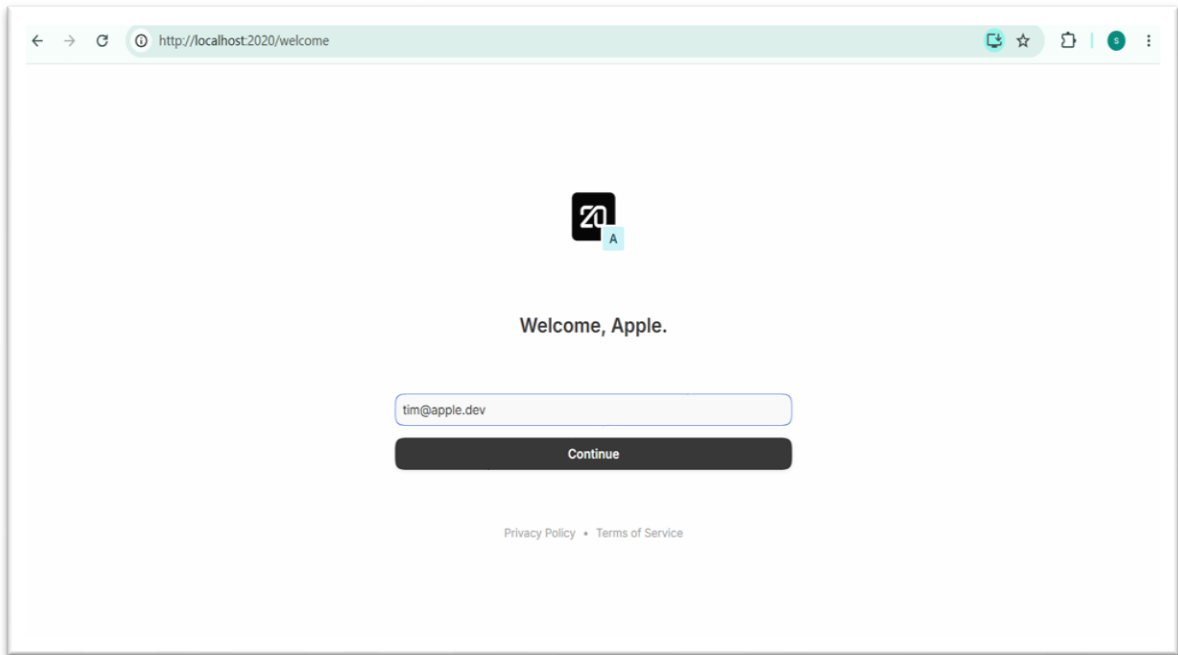
The application was successfully verified at `http://localhost:2020`, and the HTTP health check returned HTTP 200.

13. Conclusion

This task provided practical experience with Git and GitHub, Node.js, Yarn, Docker, WSL 2, local application setup, Python automation, container management, and HTTP health checks.

The Python script reduces the manual effort required to set up and start the application while also verifying that the application is actually accessible after startup.

14. Screenshots :-



```
Administrator: Windows PowerShell
=====
SETUP COMPLETED SUCCESSFULLY
=====

Application URL: http://localhost:2020
C:\msys64\home\ADMIN\devops-internship\devops-crm-project> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
7a229a41329   twentycrm/twenty-app-dev:latest    "/init"                 2 hours ago   Up 2 hours   0.0.0.0:2020->2020/tcp, [::]:2020->2020/tcp   twenty-app-dev
C:\msys64\home\ADMIN\devops-internship\devops-crm-project> yarn.cmd twenty docker:stop
twenty server stopped.
C:\msys64\home\ADMIN\devops-internship\devops-crm-project> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS   NAMES
C:\msys64\home\ADMIN\devops-internship\devops-crm-project> python setup_and_start.py
=====
twenty CRM Local Setup and Startup Automation
=====

1/5] Checking required tools...
Node.js is available
Docker is available
Yarn is available

2/5] Checking Docker...

Checking Docker health...
Docker is running correctly

3/5] Installing dependencies...

Running: yarn.cmd install
YN0000: - Yarn 4.13.0
YN0000: - Resolution step
YN0000: - Completed
YN0000: - Post-resolution validation
YN0002: my-app@workspace:. doesn't provide monaco-editor (p196b5c), requested by twenty-ui.
YN0006: Some peer dependencies are incorrectly met by your project; run yarn explain peer-requirements <hash> for details, where <hash> is the six-letter p-prefixed code
YN0000: - Completed
YN0000: - Fetch step
YN0000: - Completed in 0s 999ms

4/5] Starting Twenty server...

Running: yarn.cmd twenty docker:start
Starting existing container...
Waiting for Twenty to be ready...
> Waiting for PostgreSQL... Done
> Flushing cache... Done
> Running upgrade... Done
> Flushing cache... Done
> Database ready... Done
> Registering cron jobs... Done
Server running on http://localhost:2020

5/5] Verifying application health...

Running: yarn.cmd twenty docker:status
Status: running (healthy)
URL: http://localhost:2020
Version: v2.37.0
Login: tin@apple.dev / tin@apple.dev

Checking application health...
Application health check passed (HTTP 200)

=====
SETUP COMPLETED SUCCESSFULLY
=====

Application URL: http://localhost:2020
```

```
Administrator: Windows PowerShell
YN0000: - Post-resolution validation
YN0002: my-app@workspace:. doesn't provide monaco-editor (p196b5c), requested by twenty-ui.
YN0006: Some peer dependencies are incorrectly met by your project; run yarn explain peer-requirements <hash> for details, where <hash> is the six-letter p-prefixed code

YN0000: - Completed
YN0000: - Fetch step
YN0000: - Completed in 0s 999ms
YN0000: - Link step
YN0000: - Completed in 0s 377ms
YN0000: - Done with warnings in 1s 593ms

4/5] Starting Twenty server...

Running: yarn.cmd twenty docker:start
Starting existing container...
Waiting for Twenty to be ready...
> Waiting for PostgreSQL... Done
> Flushing cache... Done
> Running upgrade... Done
> Flushing cache... Done
> Database ready... Done
> Registering cron jobs... Done
Server running on http://localhost:2020

5/5] Verifying application health...

Running: yarn.cmd twenty docker:status
Status: running (healthy)
URL: http://localhost:2020
Version: v2.37.0
Login: tin@apple.dev / tin@apple.dev

Checking application health...
Application health check passed (HTTP 200)

=====
SETUP COMPLETED SUCCESSFULLY
=====

Application URL: http://localhost:2020
```

